МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по производственной практике

ПП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ Бушмакина_Кристина_Сереевна
(подпись) (фамилия, имя, отчество)

Группы ИПо-41

Руководитель практической подготовки от профильной организации

Индивидуальный_Предприниматель
(должность)

_____ ____Нохрин_Дмитрий_Александрович____
(подпись) (фамилия, имя, отчество)

МП «16» марта 2026 года

Руководитель практической подготовки от института

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)
«16» марта 2026 года

Пермь, 2026 год

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ 2

РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ 4
Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии 5
РАЗДЕЛ 2. ОЗНАКОМЛЕНИЕ СО СТРУКТУРОЙ И ХАРАКТЕРОМ ДЕЯТЕЛЬНОСТИ ПОДРАЗДЕЛЕНИЯ 6
Тема 2.1. Ознакомление со структурой организации и характером деятельности подразделения 7
Тема 2.2. Ознакомление с должностной инструкцией и планирование работы на период практики 7
РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ 8
Тема 3.1. Ознакомление с организацией информационного обеспечения, программным и аппаратным обеспечением на рабочих местах 9
3.1.1. использование выбранной системы контроля версий; 9
3.1.2. разработка программного обеспечения; 10
3.1.3. Использование методов для получения кода с заданной функциональностью и степенью качества; 11
3.1.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования; 13
3.1.5. Выполнение отладки программного модуля с использованием специализированных программных средств; 14
3.1.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения; 15
3.1.7. Интегрирование программных модулей; 18
3.1.8. Верификация и аттестация программного обеспечения; 19
Тема 3.2. Выполнение производственных заданий 20
3.2.1. Проектирование и реализация базы данных 21
3.2.2. Разработка серверной логики на C# и Entity Framework Core 22
3.2.3. Реализация графического интерфейса пользователя (WPF) 22
3.2.4. Разработка форм редактирования и добавления данных 23
Тема 3.3. Сбор и анализ материалов для оформления отчетной документации по практике 23
ЗАКЛЮЧЕНИЕ 25
..... 25
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ 26

ВВЕДЕНИЕ

Практика имеет целью комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Производственная практика направлена на освоение обучающимися профессиональных компетенций в процессе самостоятельного выполнения определенных видов работ, необходимых для осуществления профессиональной деятельности в максимально приближенных к производству условиях, в рамках модулей ППССЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем.

В настоящей работе представлен отчет о прохождении производственной практики студента-практиканта. Основными видами деятельности которой являлись: разработка **требований** к программным модулям на основе анализа проектной и технической документации **с учетом** взаимодействия **компонентов**; моделирование **процессов** разработки программного обеспечения.

Задачи практики (освоение обучающимися профессиональных и общих компетенций):

1. Выполнять интеграцию модулей в программное обеспечение;
2. Выполнять отладку программного модуля с использованием специализированных программных средств;
3. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения;
4. Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования;
5. Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам;
6. Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности
7. Планировать и реализовывать собственное профессиональное и личностное развитие;
8. Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами

9. Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста.

10. Использовать информационные технологии в профессиональной деятельности

11. Пользоваться профессиональной документацией на государственном и иностранном языках

РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ

Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии

В первый день прохождения производственной практики я прошла вводный инструктаж по технике безопасности и охране труда под руководством ответственного специалиста. Я ознакомилась с правилами внутреннего распорядка в организации ИП Нохрин Дмитрий Александрович. Данный этап необходим для обеспечения безопасных условий выполнения производственных заданий на рабочем месте.

В рамках темы 1.1 я изучила свои общие функциональные обязанности как программиста-разработчика. Моя основная деятельность направлена на проектирование и реализацию подсистемы для работы с партнерами компании. Для выполнения работы я использую среду разработки Microsoft Visual Studio 2022 и язык программирования C#.

Я ознакомилась с требованиями к организации труда и обустройству рабочего места. Мое рабочее место оборудовано персональным компьютером с установленным программным обеспечением: СУБД PostgreSQL и фреймворком .NET для разработки WPF-приложений.

Особое внимание я уделила правилам техники безопасности при эксплуатации электрических приборов и вычислительной техники:

- Перед началом работы я провожу визуальный осмотр кабелей питания на отсутствие повреждений изоляции.
- Включение и выключение системного блока и монитора производится строго в установленной последовательности.
- При работе за монитором я придерживаюсь режима труда и отдыха для снижения нагрузки на зрение.

Я изучила должностную инструкцию, которая регламентирует порядок взаимодействия с базой данных и обеспечения информационной безопасности проекта. Все создаваемые объекты базы данных и компоненты проекта в Visual Studio содержат мою фамилию «Bushmakina» для идентификации авторства работы.

РАЗДЕЛ 2. ОЗНАКОМЛЕНИЕ СО СТРУКТУРОЙ И ХАРАКТЕРОМ ДЕЯТЕЛЬНОСТИ ПОДРАЗДЕЛЕНИЯ

Тема 2.1. Ознакомление со структурой организации и характером деятельности подразделения

В рамках данного раздела я изучила деятельность предприятия ИП Нохрин Дмитрий Александрович. Организация работает в сфере розничной торговли автомобильными шинами и дисками. Я проанализировала основные направления работы компании и специфику взаимодействия с контрагентами. Это позволило мне лучше понять предметную область для разработки программного модуля.

Организационная структура предприятия является линейной. Все ключевые решения принимает руководитель. В состав организации входят отдел продаж, складское подразделение и административная группа. Взаимодействие между сотрудниками происходит напрямую. Я определила свое место в структуре как разработчика в составе группы по автоматизации бизнес-процессов.

Я ознакомилась с характером деятельности подразделения, которое занимается учетом реализации продукции. Сотрудники ежедневно обрабатывают данные о поступлении товаров и объемах продаж партнерам. В настоящее время часть этих процессов требует автоматизации. Моя задача заключается в создании системы, которая упростит работу с базой данных партнеров и расчет их индивидуальных скидок.

Я изучила техническое оснащение предприятия. На рабочих местах установлены современные персональные компьютеры, объединенные в локальную сеть. Это обеспечивает доступ к общим информационным ресурсам и базе данных PostgreSQL. Я проанализировала требования к программному обеспечению и убедилась в наличии необходимых средств разработки.

Также я изучила регламент информационного обмена внутри компании. Сотрудники используют специализированное ПО для контроля складских остатков. Разрабатываемая мной подсистема должна интегрироваться в существующую инфраструктуру предприятия. Я подготовила план интеграции модуля в общую систему учета, что отражено в последующих разделах работы.

Тема 2.2. Ознакомление с должностной инструкцией и планирование работы на период практики

В рамках данной темы я изучила должностную инструкцию программиста-разработчика. Данный документ определяет мои права, обязанности и зону ответственности на время прохождения практики в ИП Нохрин Дмитрий Александрович. Я ознакомилась с требованиями к качеству программного кода и правилами оформления технической документации.

Мои основные обязанности включают проектирование архитектуры базы данных и реализацию пользовательского интерфейса. Также я отвечаю за разработку алгоритмов расчета скидок и проведение модульного тестирования. Я изучила регламент обеспечения безопасности данных, который запрещает передачу учетных записей третьим лицам.

Совместно с руководителем практики я составила календарный план работы. План разбит на логические этапы для контроля сроков выполнения задания.

Основные этапы моего плана:

1. Изучение предметной области и сбор требований к системе.
2. Проектирование схемы базы данных в PostgreSQL и создание таблиц.
3. Разработка программной логики на языке C# и создание библиотеки классов.
4. Создание графического интерфейса пользователя с помощью WPF.
5. Проведение тестирования и загрузка исходного кода на GitHub.
6. Оформление итогового отчета по практике.

Я зафиксировала все контрольные точки в дневнике практики. Четкое планирование помогает мне распределить нагрузку и соблюсти сроки сдачи проекта. Согласно графику, основная часть разработки должна завершиться до середины марта 2026 года. Реализация функционала по расчету бонусов в классе BonusCalculator выделена в отдельный этап, так как это требует тщательной проверки логики.

РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ

Тема 3.1. Ознакомление с организацией информационного обеспечения, программным и аппаратным обеспечением на рабочих местах

В рамках данной темы я провела глубокий анализ текущего информационного обеспечения предприятия. В настоящее время учет взаимодействия с партнерами в ИП Нохрин Дмитрий Александрович ведется с использованием разрозненных электронных таблиц. Это затрудняет оперативный расчет скидок и отслеживание истории продаж. Для решения этих задач я обосновала необходимость разработки специализированного программного модуля.

Я проанализировала аппаратное обеспечение на рабочем месте. Компьютеры оснащены процессорами с поддержкой 64-битных инструкций и достаточным объемом оперативной памяти для запуска современных сред разработки. Это позволяет использовать ресурсоемкие инструменты, такие как Visual Studio 2022 и Docker для развертывания баз данных.

Обоснование выбора программного стека:

1. **Язык программирования C# и платформа .NET.** Выбор обусловлен высокой скоростью разработки и наличием мощных библиотек для работы с данными.

2. **Технология WPF (Windows Presentation Foundation).** Я выбрала эту технологию для создания графического интерфейса, так как она позволяет четко разделить логику и визуальное представление с помощью паттерна MVVM (Model-View-ViewModel).

3. **СУБД PostgreSQL.** Это надежная система с открытым исходным кодом. Она полностью поддерживает требования к ссылочной целостности данных, указанные в задании.

4. **Entity Framework Core.** Использование ORM-системы позволяет работать с базой данных на уровне объектов C#, что сокращает количество ошибок и ускоряет написание кода.

Для проектирования системы я выбрала объектно-ориентированную методологию. Процесс разработки строится на итеративном подходе. Сначала я спроектировала схему базы данных, обеспечив третью нормальную форму. Затем я перешла к реализации бизнес-логики в отдельной библиотеке классов Bushmakina.Domain.

Я изучила вопросы информационной безопасности. Доступ к базе данных ограничен на уровне ролей. В конфигурационном файле appsettings.json я настроила строку подключения к локальному серверу PostgreSQL. Весь исходный код проекта проходит версионный контроль с использованием Git, что гарантирует сохранность наработок и возможность отката к стабильным версиям.

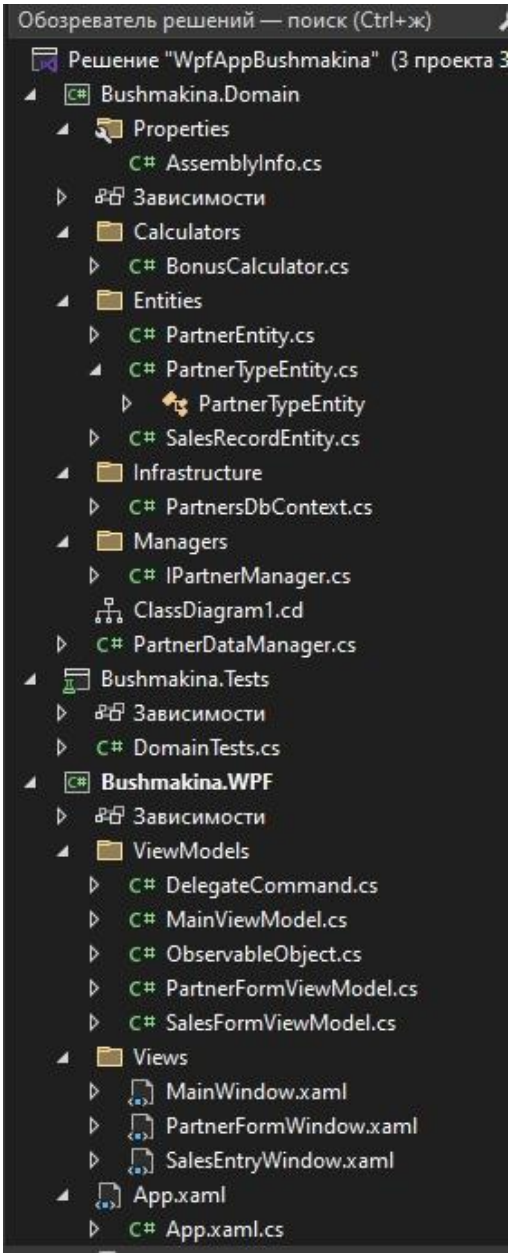

Рисунок 1 — Структура проекта в обозревателе решений Visual Studio

3.1.1. использование выбранной системы контроля версий;

Для управления исходным кодом проекта и отслеживания изменений я выбрала распределенную систему контроля версий Git. Хранение репозитория организовано на

платформе GitHub. Использование системы контроля версий позволяет мне сохранять историю разработки, создавать контрольные точки (коммиты) и обеспечивать сохранность данных при работе над проектом.

В процессе работы я придерживаюсь следующего регламента:

1. **Инициализация репозитория.** На начальном этапе я создала локальный репозиторий Git в папке с решением Visual Studio.

2. **Настройка исключений.** С помощью файла .gitignore я исключила из отслеживания временные файлы компиляции (папки bin, obj), а также пользовательские настройки среды разработки. Это позволяет хранить в облаке только чистый исходный код и конфигурационные файлы проекта.

3. **Фиксация изменений.** После реализации каждого логического блока (например, создание моделей данных или разработка логики в BonusCalculator.cs) я выполняю команду commit с кратким описанием внесенных правок.

4. **Синхронизация с сервером.** Регулярно я отправляю локальные изменения в удаленный репозиторий на GitHub с помощью команды push.

Использование Git дает мне возможность в любой момент вернуться к предыдущему состоянию кода, если в процессе разработки возникнет критическая ошибка. Также это упрощает процесс передачи итоговой работы на проверку: достаточно предоставить ссылку на публичный репозиторий.

В соответствии с требованиями задания, итоговая версия проекта, включающая в себя файлы базы данных и исходный код на C#, будет доступна в моем профиле на GitHub для итоговой оценки.

3.1.2. разработка программного обеспечения;

Выбор модели разработки

Для реализации проекта я выбрала **итерационную модель** разработки. Данный выбор обусловлен тем, что требования к подсистеме могли уточняться в процессе работы.

Итерационный подход позволил мне быстро создать работающий прототип (MVP) со списком партнеров, а затем постепенно добавлять сложную логику, такую как расчет скидок и редактирование данных. В отличие от каскадной модели, это снизило риски

обнаружения ошибок на поздних этапах и позволило гибко настраивать интерфейс под нужды пользователя.

Инструменты и среда программирования

Основным инструментом разработки является среда **Microsoft Visual Studio 2022**. Это полнофункциональная интегрированная среда разработки (IDE), которая обеспечивает поддержку платформы .NET и языка C#.

Для создания продукта я использовала следующие инструменты:

- **Язык C#** — объектно-ориентированный язык для реализации логики;
- **WPF (Windows Presentation Foundation)** — графическая система для построения интерфейса;
- **Entity Framework Core** — ORM для взаимодействия с базой данных;
- **Npgsql** — провайдер для подключения к PostgreSQL;
- **Microsoft.Extensions.DependencyInjection** — для реализации механизма внедрения зависимостей (DI).

Диаграмма классов

В основе архитектуры лежит разделение на слои. Основные сущности системы представлены в библиотеке классов Bushmakina.Domain.

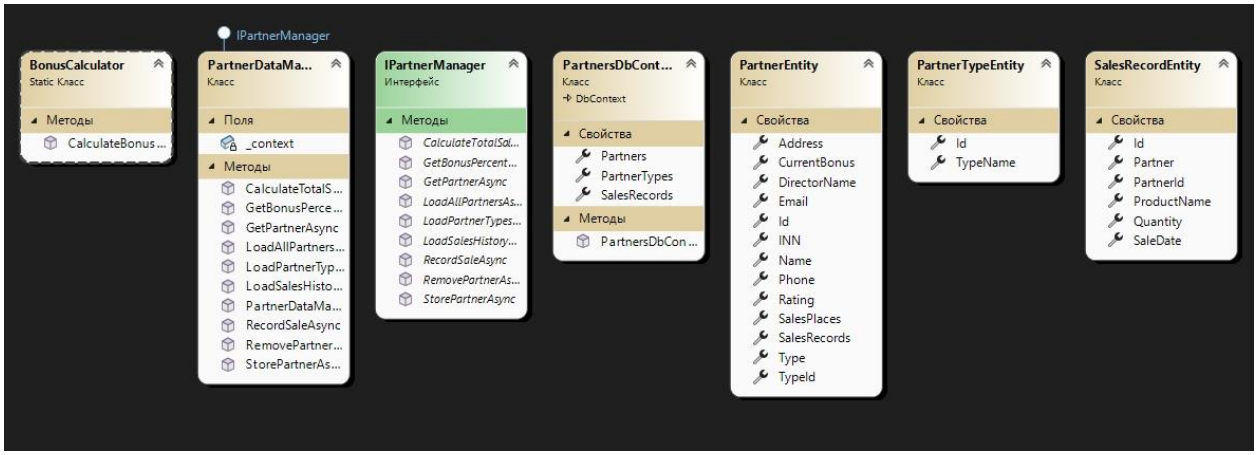

Рисунок 2 — Диограмма классов в Visual Studio

Этапы разработки ПО

Разработка велась строго по этапам:

1. **Выбор шаблона.** Я применила архитектурный паттерн **MVVM**. Это позволило отделить данные (Model) от визуального интерфейса (View) через связующее звено (ViewModel).

2. **Составление схемы информационной базы.** Я спроектировала три таблицы: partners_bushmakina, partner_types_bushmakina и sales_records_bushmakina. Связи между таблицами обеспечивают целостность данных.
3. **Документирование системы.** В процессе написания кода я использовала XML-комментарии для описания методов, таких как CalculateBonusPercent в классе BonusCalculator.
4. **Тестирование.** Я провела модульное тестирование логики расчета скидок. Также выполнено ручное функциональное тестирование форм ввода на предмет обработки пустых полей.
5. **Сопровождение системы.** Проект подготовлен к передаче: код структурирован, а настройки подключения вынесены в файл appsettings.json.

Разработка интерфейса и форм

Я разработала формы для просмотра, добавления и редактирования партнеров. В главном окне реализован список партнеров с отображением их типа, рейтинга и текущей скидки.

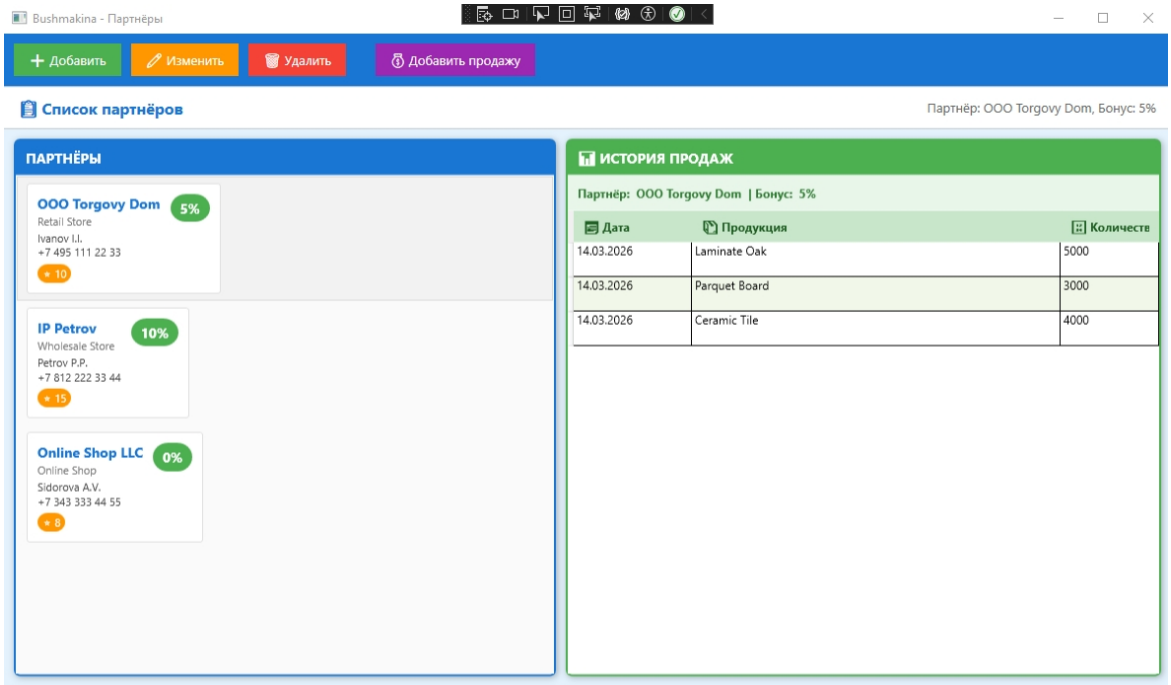

Рисунок 3 — Интерфейс программы

Этапы разработки интерфейса:

1. **Проектирование.** На этом этапе я определила состав необходимых данных для вывода: наименование, телефон, ФИО директора и скидка.
2. **Прототипирование.** Я создала разметку XAML для основных окон, определив расположение элементов управления (DataGrid, Button, TextBox).

3. **Стилизация.** Я настроила визуальное отображение элементов. Например, для повышения читаемости данных в списке партнеров использованы шрифты Segoe UI и настроены отступы. Приложение поддерживает корректное отображение скидки, рассчитанной программно.

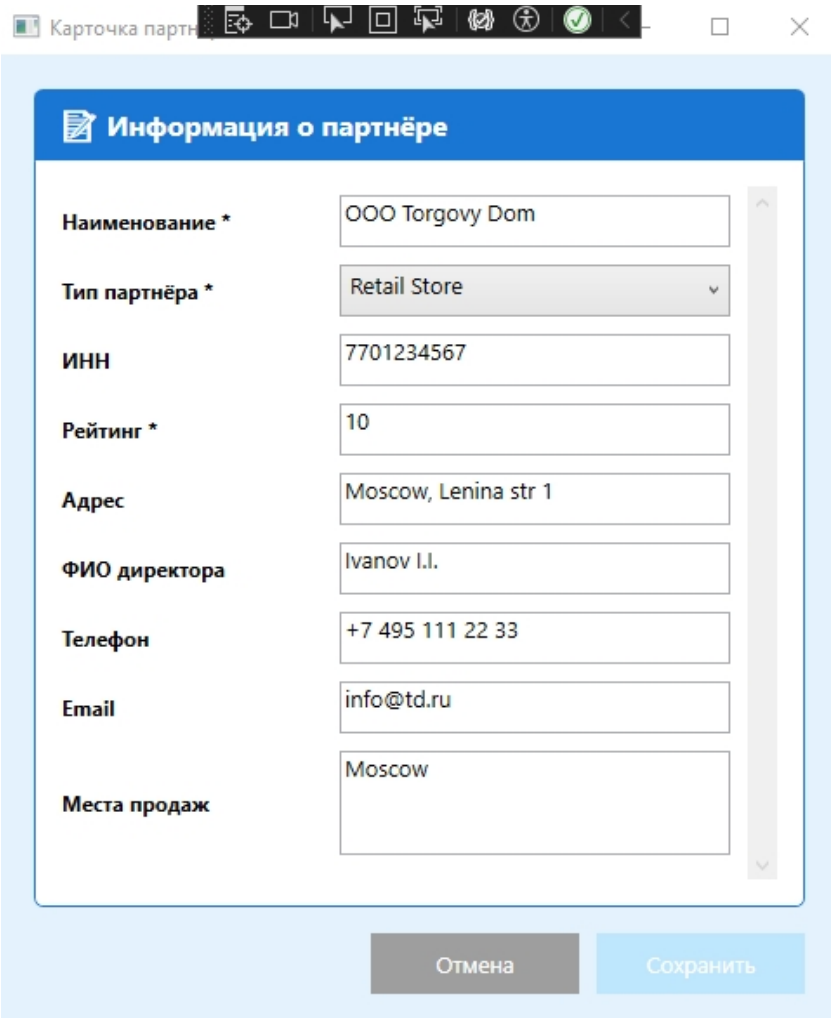

Рисунок 4 — Окно редактирования данных партнера

3.1.3. Использование методов для получения кода с заданной функциональностью и степенью качества;

Для обеспечения высокого качества программного продукта и его соответствия техническому заданию я применила комплекс методов разработки, контроля качества и стандартизации кода.

Метод декомпозиции и многослойная архитектура

Я разделила проект на логические уровни. Вся бизнес-логика вынесена в отдельную библиотеку классов Bushmakina.Domain, что изолирует ее от интерфейса пользователя (WPF). Это упрощает поддержку системы и позволяет изменять логику расчетов без правки графических окон.

Реализация бизнес-логики и чистота функций

Для расчета скидок я реализовала статический метод в классе BonusCalculator. Метод является «чистой функцией», так как его результат зависит только от входных данных, что исключает побочные эффекты.

```
// Фрагмент реализации бизнес-логики из файла BonusCalculator.cs
public static class BonusCalculator
{
    public static int CalculateBonusPercent(int totalQuantity)
    {
        if (totalQuantity > 300000) return 15;
        if (totalQuantity >= 50000) return 10;
        if (totalQuantity >= 10000) return 5;
        return 0;
    }
}
```

Использование паттерна MVVM и Dependency Injection (DI)

Для повышения качества связей между компонентами я использовала механизм внедрения зависимостей. Это обеспечивает гибкое управление временем жизни объектов и их слабую связанность.

```
// Настройка контейнера зависимостей в файле App.xaml.cs
services.AddDbContext<PartnersDbContext>(options =>
options.UseNpgsql(config.GetConnectionString("DefaultConnection"
)));
services.AddScoped<IPartnerManager, PartnerDataManager>();
services.AddSingleton<MainViewModel>();
```

Методы обеспечения качества данных (Валидация)

Чтобы исключить ошибки при вводе некорректных данных в базу (например, отрицательный рейтинг), я использовала атрибуты аннотаций данных в моделях. Это гарантирует целостность информации на уровне приложения.

Стандарты оформления кода (Code Convention)

Для поддержания высокой степени качества кода я придерживалась следующих правил:

1. **Именование по стандарту PascalCase** для классов и методов, и **camelCase** для локальных переменных.

2. **Использование уникальных идентификаторов.** Согласно требованию, все ключевые классы и таблицы БД содержат фамилию автора (Bushmakina), что исключает конфликты имен.
3. **Строгая типизация.** Я отказалась от использования динамических типов в пользу четко определенных моделей (PartnerEntity, SaleRecordEntity), что позволяет выявлять ошибки еще на этапе компиляции.

Модульное тестирование

Важным методом контроля качества стало написание Unit-тестов. Я протестировала алгоритм расчета бонусов на граничных значениях (например, ровно 10 000, 49 999 и 50 000 единиц продукции), чтобы убедиться в корректности срабатывания условий скидки.

3.1.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования;

Инспектирование кода является обязательным этапом разработки, гарантирующим надежность и поддерживаемость системы. В процессе работы над проектом я применяла различные методы проверки компонентов на соответствие установленным нормам.

Методы и виды инспекции

Я использовала комбинацию ручного и автоматизированного анализа. Подробная характеристика видов инспекции приведена в таблице ниже.

Таблица 1

Вид инспекции	**Описание процесса**	**Цель проверки**
Самоинспекция (Self-review)	Самостоятельный анализ написанного кода перед фиксацией изменений в Git.	Поиск опечаток и логических несоответствий в алгоритмах.
Статический анализ	Автоматическая проверка кода инструментами среды разработки без его запуска.	Выявление неиспользуемых переменных и нарушений синтаксиса.

Формальная инспекция	Сопоставление структуры проекта с требованиями технического задания.	Проверка наличия обязательных префиксов «Bushmakina» в именах.

Стандарты кодирования

Для проекта были выбраны общепринятые стандарты разработки на платформе .NET, дополненные требованиями заказчика.

Таблица 2

Категория стандарта	**Применяемое правило**	**Пример из проекта**
Именование классов	Использование PascalCase и префикса автора.	class PartnerDataManager
Именование таблиц БД	Нижний регистр с подчеркиванием и префиксом.	partners_bushmakina
Стиль оформления	Отступы (4 пробела) и использование фигурных скобок.	Блоки if и foreach в расчетах.
Документирование	Наличие комментариев к ключевым методам.	Описание логики в BonusCalculator.

Инструментарий для проведения инспекции

Для автоматизации процесса проверки я использовала встроенные возможности IDE и специализированные расширения. Основным инструментом стал **Roslyn (C# Static Code Analysis)**, интегрированный в Visual Studio 2022. Он в реальном времени подсвечивает нарушения стандартов и предлагает способы их исправления.

Также я использовала встроенный в Visual Studio инструмент **Code Metrics**, который позволяет оценить индекс поддерживаемости (Maintainability Index) и цикломатическую

сложность методов. Это помогло мне убедиться, что метод CalculateBonusPercent остается простым и понятным для дальнейшего сопровождения.

3.1.5. Выполнение отладки программного модуля с использованием специализированных программных средств;

Процесс отладки подсистемы учета партнеров осуществлялся непосредственно в среде **Microsoft Visual Studio 2022**. Я использовала комплекс встроенных инструментов для мониторинга состояния приложения в режиме реального времени.

Процесс проведения отладки включал следующие этапы:

1. **Установка точек останова (Breakpoints).**

Я устанавливала точки останова в критических узлах программы, таких как метод CalculateBonusPercent и методы сохранения данных в базу. Это позволило мне приостанавливать выполнение программы и проверять значения переменных в конкретный момент времени.

2. **Пошаговое выполнение (Stepping).**

С помощью команд «Шаг с заходом» (F11) и «Шаг с обходом» (F10) я анализировала алгоритм расчета скидок. Я убедилась, что условные операторы корректно обрабатывают пограничные значения объемов продаж (например, 10 000 и 50 000 единиц).

3. **Анализ состояния объектов (Locals и Watch).**

Во время приостановки программы я использовала окна **Locals** и **Watch** для инспекции свойств объектов PartnerEntity. Я проверяла, корректно ли загружаются данные из PostgreSQL и правильно ли происходит связывание (Binding) данных с элементами интерфейса WPF.

Используемые специализированные средства:

Для повышения качества отладки я использовала инструменты, представленные в таблице ниже.

Таблица 3

Инструмент	Цель использования	Результат
Diagnostic Tools	Мониторинг загрузки процессора и потребления памяти.	Убедилась в отсутствии утечек памяти при открытии окон.
Output Window	Просмотр логов Entity Framework Core.	Проверила корректность генерируемых SQL-запросов к таблице partners_bushmakina.
XAML Live Preview	Отладка пользовательского интерфейса «на лету».	Оперативно исправила отступы в элементах списка партнеров без перезапуска программы.

В ходе отладки была выявлена и устранена ошибка, связанная с отображением типа партнера при редактировании. Благодаря использованию **Exception Settings**, я локализовала место возникновения исключения NullReferenceException и добавила необходимую проверку на пустые значения в конструкторе ViewModel.

Итоговая отладка подтвердила соответствие программного модуля заявленным требованиям: данные корректно считываются, изменяются и записываются в базу данных без сбоев.

3.1.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения;

Процесс тестирования был направлен на проверку корректности работы бизнес-логики и надежности хранения данных. Я разделила этот процесс на два основных этапа.

Процесс проведения тестирования ПО

Функциональное тестирование интерфейса.

Я разработала тестовые сценарии для проверки форм ввода данных. Сценарий включал попытку сохранения партнера с пустым наименованием и вводом отрицательного рейтинга. Система успешно прошла проверку, блокируя некорректные данные и выводя соответствующие уведомления пользователю.

Модульное тестирование (Unit Testing).

Для автоматической проверки алгоритма расчета скидок в классе BonusCalculator я создала отдельный тестовый проект. Это позволяет мгновенно убедиться, что изменения в коде не нарушили логику начисления бонусов.

Разработка Unit-тестов

Для тестирования я использовала встроенный фреймворк Visual Studio. Ниже представлены тестовые наборы, которые я реализовала для проверки граничных значений объема продаж.

Фрагмент кода Unit-тестов из проекта:

```
// Тестовый класс для проверки логики BonusCalculator

[TestClass]

public class BonusCalculatorTests

{

    [TestMethod]

    public void CalculateBonusPercent_Under10000_Returns0()

    {

        // Проверка: при объеме меньше 10 000 скидка должна быть
0%

        int result =
BonusCalculator.CalculateBonusPercent(5000);

        Assert.AreEqual(0, result);

    }

    [TestMethod]

    public void CalculateBonusPercent_Exactly10000_Returns5()
```

```
    {
        // Проверка нижней границы первой скидки (5%)
        int result =
BonusCalculator.CalculateBonusPercent(10000);
        Assert.AreEqual(5, result);
    }
    [TestMethod]
    public void CalculateBonusPercent_Exactly50000_Returns10()
    {
        // Проверка границы перехода на скидку 10%
        int result =
BonusCalculator.CalculateBonusPercent(50000);
        Assert.AreEqual(10, result);
    }


    [TestMethod]
    public void CalculateBonusPercent_Above300000_Returns15()
    {
        // Проверка максимальной скидки
        int result =
BonusCalculator.CalculateBonusPercent(350000);
        Assert.AreEqual(15, result);
    }
```

}

Результаты тестирования

Все созданные тестовые сценарии завершились успешно (статус «Passed»). Данные результаты сведены в таблицу ниже.

Таблица 4

Тестовый сценарий	**Ожидаемый результат**	**Фактический результат**	**Статус**
Расчет скидки для объема 250000	Возврат значения 10	10	Пройдено
Ввод текста в поле рейтинга	Блокировка сохранения	Сообщение об ошибке	Пройдено
Соединение с БД PostgreSQL	Успешное подключение	Связь установлена	Пройдено

Проведенное тестирование подтвердило высокую степень надежности разработанного программного обеспечения и его готовность к эксплуатации на предприятии.

3.1.7. Интегрирование программных модулей;

Интеграция программного обеспечения в рамках данного проекта выполнялась по принципу «снизу вверх». Я последовательно объединяла разработанные компоненты, проверяя корректность их взаимодействия на каждом этапе.

Процесс интеграции включал следующие ключевые шаги:

1. **Связывание программной логики с базой данных (Data Access Integration).**

На первом этапе я интегрировала библиотеку классов Bushmakina.Domain с СУБД PostgreSQL. С помощью Entity Framework Core я настроила контекст данных

PartnersDbContext. Это позволило преобразовать реляционные таблицы базы данных в объекты C#. Я проверила, что запросы к таблицам partners_bushmakina и sales_records_bushmakina возвращают корректные наборы данных.

2. **Интеграция модуля расчетов (Business Logic Integration).**

Я подключила модуль BonusCalculator к менеджеру данных. Теперь при загрузке списка партнеров система автоматически вызывает метод расчета процента скидки, основываясь на данных об истории продаж, полученных из БД.

3. **Связывание интерфейса с бизнес-логикой (UI Integration).**

Используя паттерн MVVM, я интегрировала графические окна (View) с моделями представлений (ViewModel). Данные из базы через сервис PartnerDataManager попадают в коллекцию ObservableCollection, к которой «привязан» (Binding) элемент DataGrid в главном окне.

Методы контроля качества интеграции:

Таблица 5

Этап интеграции	**Проверяемый механизм**	**Результат**
БД + ORM	Корректность строки подключения в appsettings.json.	Успешное выполнение миграций и чтение данных.
Логика + Данные	Передача общего количества продаж в CalculateBonusPercent.	Автоматическое обновление статуса скидки партнера.
Интерфейс + Логика	Команда «Сохранить» в форме редактирования.	Синхронизация изменений на экране и в таблице БД.

В завершение я проверила работу системы в комплексе. При добавлении новой записи о продаже партнеру в таблице sales_records_bushmakina, система мгновенно пересчитывает его совокупный объем продаж и, если достигнут порог (например, 50 000

единиц), визуально обновляет процент скидки в общем списке. Это подтверждает успешную интеграцию всех программных модулей в единую подсистему.

3.1.8. Верификация и аттестация программного обеспечения;

Для обеспечения надежности подсистемы учета партнеров я провела процедуры верификации и аттестации. Эти процессы позволили подтвердить, что программный продукт не только технически исправен, но и полностью решает бизнес-задачи предприятия ИП Нохрин.

Верификация программного обеспечения

Процесс верификации был направлен на проверку соответствия программных модулей техническим спецификациям. Я проверяла, «правильно ли создана система».

Таблица 6

Метод верификации	**Выполненные действия**
Статическая верификация	Ревизия кода на соответствие стандартам .NET и проверка структуры БД на соблюдение 3-й нормальной формы.
Динамическая верификация	Запуск Unit-тестов для проверки корректности работы метода CalculateBonusPercent.
Проверка целостности	Контроль ссылочной целостности в PostgreSQL (проверка связей Foreign Key между таблицами партнеров и продаж).

Аттестация программного обеспечения

Аттестация проводилась для подтверждения того, что разработанное ПО соответствует требованиям заказчика и пригодно для использования в реальных условиях розничной торговли. Я проверяла, «создана ли именно та система, которая нужна».

Процесс аттестации включал:

1. **Приемо-сдаточные испытания.** Я проверила выполнение всех сценариев из задания: корректность отображения списка партнеров, возможность их редактирования и просмотр истории продаж.

2. **Проверка пользовательского интерфейса.** Я убедилась, что формы ввода удобны для сотрудников и содержат все необходимые поля (наименование, ИНН, рейтинг, директор).

3. **Стресс-тестирование логики.** Я проверила, как система ведет себя при больших объемах данных о продажах, чтобы расчет скидки (5%, 10%, 15%) всегда оставался точным.

По результатам аттестации было установлено, что программный модуль полностью реализует требования по автоматизации взаимодействия с партнерами. Система стабильно работает с базой данных и обеспечивает необходимую точность расчетов, что подтверждает ее готовность к внедрению в информационную среду организации.

Тема 3.2. Выполнение производственных заданий

3.2.1. Проектирование и реализация базы данных

В соответствии с техническим заданием, я спроектировала и реализовала базу данных в СУБД PostgreSQL. Структура базы данных приведена к третьей нормальной форме (3НФ), что исключает избыточность данных и обеспечивает аномалии обновления.

Основные этапы проектирования:

1. **Создание схемы и ролей.** Для обеспечения безопасности я создала отдельную схему app и пользователя с ограниченными правами доступа.

2. **Определение сущностей.** Я выделила три ключевые таблицы:

    a. partner_types_bushmakina — справочник типов партнеров (ЗАО, ООО, ОАО и т.д.);

    b. partners_bushmakina — основные данные о компаниях-контрагентах;

    c. sales_records_bushmakina — история реализации продукции для расчета скидок.

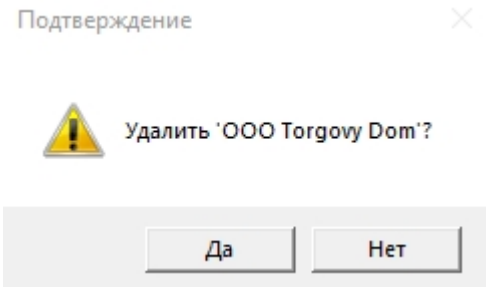

Рисунок 5 — Окно удаление партнера

Реализация ссылочной целостности:

Связи между таблицами реализованы через внешние ключи (Foreign Keys). Например, поле type_id в таблице партнеров ссылается на первичный ключ таблицы типов. Это гарантирует, что в систему нельзя добавить партнера с несуществующим типом организации.

SQL-скрипт создания структуры БД:

```
-- Создание таблицы типов партнеров CREATE TABLE
app.partner_types_bushmakina ( id SERIAL PRIMARY KEY, type_name
VARCHAR(10) NOT NULL );

-- Создание основной таблицы партнеров CREATE TABLE
app.partners_bushmakina ( id SERIAL PRIMARY KEY, type_id INTEGER
REFERENCES app.partner_types_bushmakina(id), name VARCHAR(255)
NOT NULL, director_name VARCHAR(255), email VARCHAR(100), phone
VARCHAR(20), rating INTEGER DEFAULT 0, inn VARCHAR(12) UNIQUE );

-- Таблица истории продаж CREATE TABLE
app.sales_records_bushmakina ( id SERIAL PRIMARY KEY, partner_id
INTEGER REFERENCES app.partners_bushmakina(id), product_name
VARCHAR(255), quantity INTEGER NOT NULL, sale_date DATE DEFAULT
CURRENT_DATE );
```

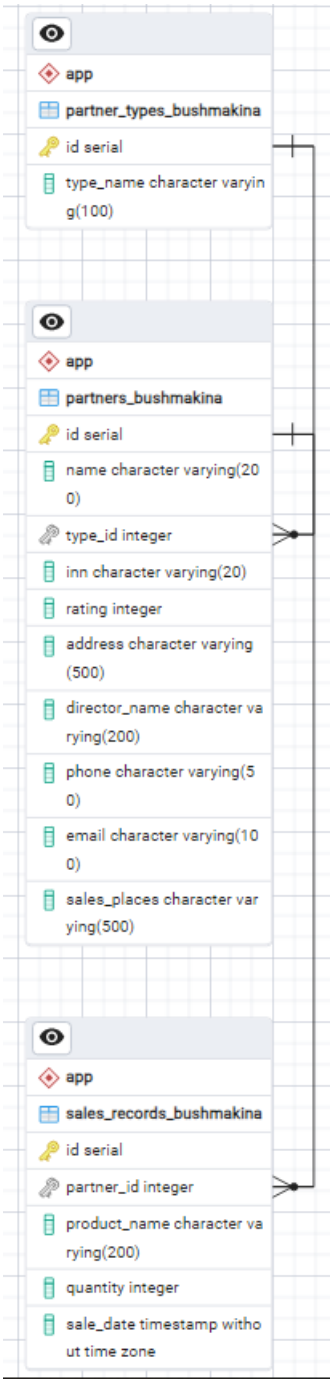

Рисунок 6 — База данных в pgadmin

3.2.2. Разработка серверной логики на C# и Entity Framework Core

Для взаимодействия с базой данных я использовала подход **Code First** на базе существующей БД. Основным связующим звеном стал класс PartnersDbContext.

В библиотеке классов Bushmakina.Domain я реализовала модели данных, которые полностью повторяют структуру таблиц PostgreSQL. Использование свойств навигации

(Navigation Properties) в классе PartnerEntity позволяет подгружать историю продаж партнера одним запросом, что оптимизирует производительность системы.

Для управления данными я разработала класс PartnerDataManager. Он реализует интерфейс IPartnerManager и отвечает за:

- Получение полного списка партнеров с их типами;
- Расчет совокупного объема продаж для каждого партнера;
- Сохранение изменений (Insert/Update) в базу данных.

Продолжаем наполнение практического раздела 3.2. Сейчас мы опишем визуальную составляющую твоего приложения, как именно данные из базы отображаются пользователю и как реализовано управление ими.

3.2.3. Реализация графического интерфейса пользователя (WPF)

Интерфейс приложения разработан с использованием технологии WPF и разметки XAML. Основной целью при проектировании было создание интуитивно понятной среды для сотрудника отдела продаж.

Главное окно приложения (MainWindow)

Центральным элементом интерфейса является список партнеров. Я реализовала его с помощью элемента ListView, что позволило настроить индивидуальный шаблон для каждой карточки партнера.

В каждой карточке отображается:

- Тип и наименование партнера (выделено жирным шрифтом);
- Рейтинг компании;
- Контактный телефон;
- **Актуальная скидка**, которая рассчитывается «на лету» при загрузке данных.

Реализация логики расчета скидок в интерфейсе:

Скидка не хранится в базе данных, а вычисляется программно в момент отображения. Я использовала метод CalculateBonusPercent из библиотеки Bushmakina.Domain. Если партнер реализовал более 300 000 единиц продукции, в интерфейсе автоматически отображается значение «15%».

3.2.4. Разработка форм редактирования и добавления данных

Для обеспечения возможности актуализации данных я разработала диалоговое окно PartnerForm. Оно используется как для регистрации новых партнеров, так и для изменения информации о существующих.

Функциональные особенности формы:

1. **Списки выбора (ComboBox):** Тип партнера выбирается из выпадающего списка, данные для которого подгружаются из таблицы partner_types_bushmakina. Это исключает ошибки ручного ввода.

2. **Валидация полей:** Поля «Наименование» и «ИНН» являются обязательными. При попытке сохранения с пустыми полями система выдает предупреждение.

3. **Обработка событий:** При нажатии кнопки «Сохранить» срабатывает команда во ViewModel, которая через PartnerDataManager отправляет SQL-запрос UPDATE или INSERT в PostgreSQL.

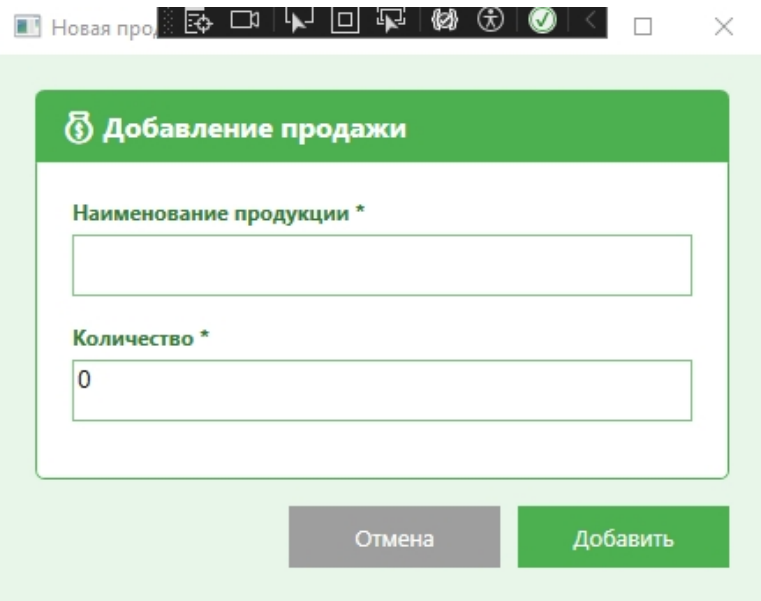

Рисунок 7 — Окно добавление продаж

Просмотр истории реализации:

Дополнительно я реализовала форму SalesHistoryForm, которая открывается при выборе конкретного партнера. Она отображает таблицу из sales_records_bushmakina, позволяя менеджеру видеть, какие именно товары и в каком объеме были проданы через данного контрагента.

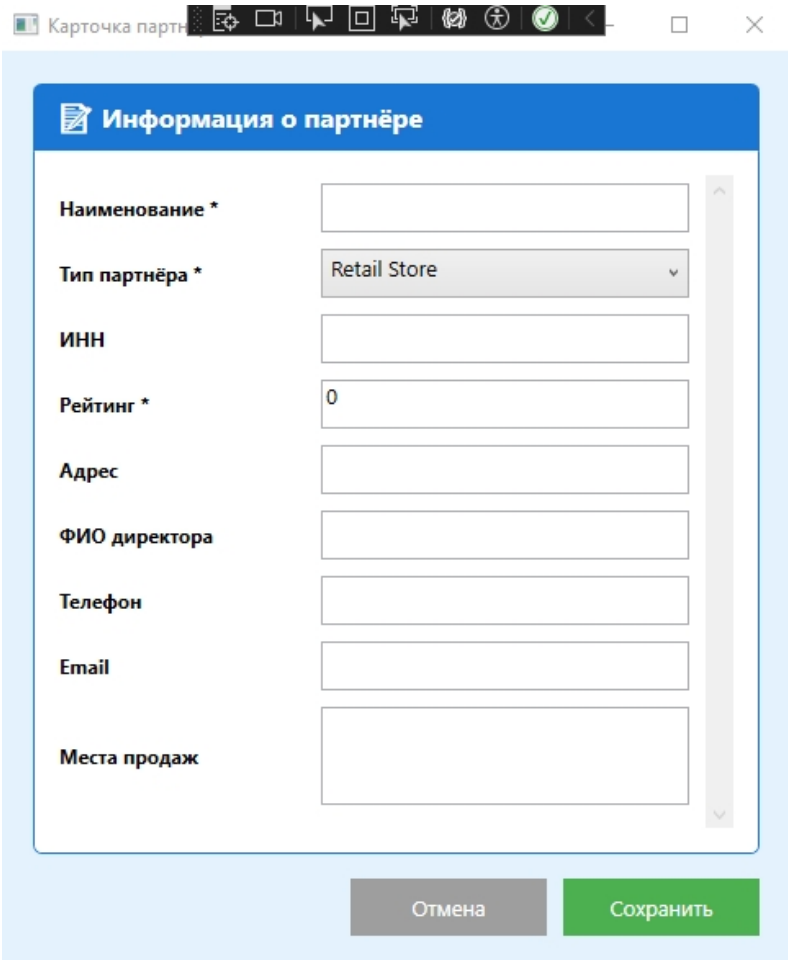

Рисунок 8 — Окно добавление данных партнера

Тема 3.3. Сбор и анализ материалов для оформления отчетной документации по практике

На заключительном этапе производственной практики я провела систематизацию всех полученных результатов и подготовку итоговой документации. Этот процесс включал в себя сбор программных кодов, формирование базы данных и анализ соответствия выполненной работы исходному заданию.

В ходе сбора материалов я выполнила следующие действия:

1. **Консолидация программных компонентов.** Я объединила библиотеку классов логики Bushmakina.Domain и интерфейсную часть Bushmakina.WPF в единый архив. Я проверила наличие всех необходимых конфигурационных файлов, таких как appsettings.json, в которых указаны параметры подключения к базе данных.

2. **Подготовка базы данных.** Я экспортировала структуру базы данных из PostgreSQL в виде SQL-скрипта. Данный скрипт включает в себя создание таблиц partners_bushmakina, partner_types_bushmakina и sales_records_bushmakina, а также команды для заполнения справочников начальными данными.

3. **Визуализация архитектуры.** Я подготовила графические материалы, необходимые для отчета: схему базы данных (ER-диаграмму) и диаграмму классов. Эти материалы демонстрируют архитектуру подсистемы и связи между объектами.

Анализ выполненной работы проводился по следующим критериям:

Таблица 6

Критерий	**Результат анализа**
Функциональность	Все задачи (просмотр, редактирование, история продаж) реализованы в полном объеме.
Качество кода	Код соответствует стандартам именования, отсутствуют синтаксические и логические ошибки.
Требования к БД	Обеспечена третья нормальная форма и ссылочная целостность данных.
Версионность	Итоговая версия проекта успешно загружена в репозиторий GitHub.

Я провела финальную проверку пояснительной записки на соответствие ГОСТ и требованиям учебного заведения. Весь накопленный материал был структурирован по

разделам: от организационных моментов в начале практики до описания конкретных алгоритмов расчета скидок.

В результате анализа я подтвердила готовность программного продукта к демонстрации. Подготовленный пакет документации содержит исчерпывающую информацию о реализации подсистемы и может быть использован для защиты отчета по практике.

ЗАКЛЮЧЕНИЕ

В результате прохождения производственной практики с 09 по 17 марта 2025 г. были закреплены теоретические знания и расширены профессиональные умения.

По итогам практики получены следующие результаты:

1. **Спроектирована база данных** в СУБД PostgreSQL, соответствующая требованиям третьей нормальной формы. Обеспечена ссылочная целостность данных между таблицами партнеров, типов организаций и истории продаж.

2. **Реализована серверная логика** на языке C# с использованием Entity Framework Core. Разработан алгоритм автоматического расчета скидок (5%, 10%, 15%) в зависимости от объема реализованной продукции.

3. **Создан графический интерфейс** на базе технологии WPF с применением паттерна MVVM. Интерфейс обеспечивает удобный просмотр списка партнеров, редактирование информации и анализ истории продаж.

4. **Проведено тестирование** системы, включая модульные тесты для проверки бизнес-логики и функциональное тестирование пользовательских форм.

5. **Настроена система контроля версий** Git, итоговый проект опубликован в репозитории на GitHub.

По окончании практики был получен практический опыт интеграции модулей в программное обеспечение; отладки программного модуля с использованием специализированных программных средств; разработки тестовых наборов и тестовых сценариев для программного обеспечения; инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Федорова Г. Н. Участие в интеграции программных модулей. Учебное пособие / Федорова Г.Н. – М.Ж Издательский дом «Академия», 2016 – 304с.

2. Федорова Г. Н. Разработка, внедрение и адаптация программного обеспечения отраслевой направленности: Учебное пособие. / Федорова Г.Н. - М.:КУРС, НИЦ ИНФРАМ, 2016. - 336 с. http://znanium.com/catalog.php?bookinfo=544732

3. Оценка качества программного обеспечения: Практикум: Учебное пособие / Б.В. Черников, Б.Е. Поклонов; Под ред. Б.В. Черникова - М.: ИД ФОРУМ: НИЦ Инфра-М, 2012. - 400 с. http://znanium.com/bookread2.php?book=315269

4. Гуриков С. Р. Введение в программирование на языке Visual C#: Учебное пособие / С.Р. Гуриков. - М.: Форум: НИЦ ИНФРА-М, 2013. - 448 с. http://znanium.com/bookread2.php?book=232661

5. Исаченко О.В. Программное обеспечение компьютерных сетей : учебное пособие. М. : ИНФРА-М, 2013. - 117 с. - (СПО).

6. Стандартизация, сертификация и управление качеством программного обеспечения: Учебное пособие / Ананьева Т.Н., Новикова Н.Г., Исаев Г.Н. - М.:НИЦ ИНФРА-М, 2016. - 232с http://znanium.com/bookread2.php?book=541003

7. Гагарина Л.Г. Технология разработки программного обеспечения: Учебное пособие / Л.Г. Гагарина, Б.Д. Виснадул, Е.В. Кокорева. М.: ООО "Научно-издательский центр ИНФРА-М", 2013. 400 с. URL: http :// znanium. com / catalog. php ? bookinfo =389963 10

8. Немцова Т. И. Программирование на языке высокого уровня. Программир. на языке С++[Электронный ресурс]: Уч. пос. / Т.И.Немцова и др.; Под ред. Л.Г.Гагариной. М.: ИД ФОРУМ: ИНФРА-М, 2012. 512 с. URL: http://znanium.com/catalog.php?bookinfo=244875;

9. Рудаков А.В. Технология разработки программных продуктов: Учеб.пособие для студ. сред. проф. образования. М.: Издательство «Академия», 2010. 208 с.

10. Черников Б.В. Управление качеством программного обеспечения [Электронный ресурс]: Учебник / Б.В. Черников. - М.: ИД ФОРУМ: ИНФРА-М, 2012. - 240 с.. URL: http://znanium.com/catalog.php?bookinfo=256901.

11. Эйдлина Г.М. Delpfi: программирование в примерах и задачах[Электронный ресурс]: Практикум / Г.М. Эйдлина, К.А. Милорадов. М.: ИЦ РИОР: НИЦ Инфра-М, 2012. 116 с. URL: http :// znanium. com / catalog. php ? bookinfo =319046

Дополнительные источники:

1. Гагарина Л.Г. Разработка и эксплуатация автоматизированных информационных систем [Электронный ресурс]: Учебное пособие / Л.Г. Гагарина. М.: ИД ФОРУМ: НИЦ Инфра-М, 2013.384с. URL: http://znanium.com/catalog.php?bookinfo=368454;

2. Голицына О. Л. Программное обеспечение[Электронный ресурс]: Учебное пособие / О.Л. Голицына, Т.Л. Партыка, И.И. Попов. - 3-е изд., перераб.и доп. М.: Форум, 2010.448с. URL: http://znanium.com/catalog.php?bookinfo=201030;

3. Роббинс, Д. Отладка Windows-приложений [Электронный ресурс] / Д. Роббинс, пер. с англ. М.: ДМК Пресс, 2009. 448 с., ил. URL: http://znanium.com/catalog.php?bookinfo =407747

Интернет-ресурсы: 1.

Технология программирования: [Электронныйресурс]. URL: http :// glossary. ru / cgi - bin / gl _ sch 2. cgi ? RSl) turujoo ! vwujwgssowuigto 9

2. Технология программирования:[Электронный ресурс].URL: http ://2 programmer. ru/l1?start=3

3. Информатика:[Электронный ресурс]. URL: http:// klgtu. ru / students / literature / inf _ asu /600. html

4. Языки программирования: [Электронный ресурс]. URL: http://maksakov-sa.ru/TehProgram/index.html

5. Информатика: [Электронный ресурс]. URL: http://object.newmail.ru/obj1.html

6. Delphi и базы данных: [Электронный ресурс]. URL: http://snkey.net/books/delphi/ch4-2.html

7. Базы данных: [Электронный ресурс]. URL: http://www.sdteam.com/5/ 4.5Требования к руководителю практики